

LS100 Guide

Python Environment Setup & Introduction to Object Detection Using YOLO with Jupyter Notebook

Souvik Mandal, Ph.D.

Introduction: What is YOLO?

YOLO (You Only Look Once) is a state-of-the-art computer vision model used for image classification (which tells you *what* is in an image), object detection (*what* is in the image and *where* it is by drawing a box around it) and pose estimation (giving the coordinates of human body parts).

YOLO is designed to be incredibly fast and accurate, making it capable of detecting objects in real-time video. For this course, we will use the `ultralytics` library, which makes using YOLO very simple, even if you are new to Python.

Setting Up Your Environment

While creating a separate environment for each task isn't mandatory, it's good practice to isolate dependencies and avoid version conflicts. Follow the steps below to set up a dedicated environment for **Object Detection using YOLO**. In case you want to use some other model later, you can safely delete this environment from your computer.

Step 1: Open your Terminal and ensure you are in the default system environment

Open your **Terminal** and make sure you are not in any virtual environment.

- If your command prompt already starts with your username, the hostname (your computer's name), or the current working directory (i.e., if you **do not** see a name in parentheses), you are in your default system environment.
- If your command prompt starts with a **name in parentheses**, like `(base)`, `(myenv)`, or `(venv)`, you are in a virtual environment and need to exit.
- If you see `(base)`, you are in the base Conda environment, type: `conda deactivate` and press Enter.
- If you see `(venv)` or something similar, you are inside a virtual environment. Type: `deactivate` and press Enter.

Step 2: Install Homebrew (For macOS Users Only)

If you are on Windows, skip to Step 3.

What is Homebrew and why do you need it? Homebrew is a "package manager" for macOS. You can think of it as an App Store that works entirely inside your Terminal. Mac computers do not come with a built-in way to easily download programming tools from the command line. By installing Homebrew, you can easily and safely download software tools (like the specific version of Python we need) just by typing a simple text command, rather than hunting for manual installers on the internet.

If you do not have Homebrew installed, paste the following command into your terminal and press Enter (follow the on-screen instructions):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 3: Install Python 3.12

YOLO requires a modern version of Python (3.8 or newer). We will use Python 3.12.

- **macOS:** `brew install python@3.12`
- **Windows:** Download the installer from python.org or use `winget install Python.Python.3.12`.

Step 4: Create a virtual environment

We will create a specific folder to hold the tools for this project.

1. Navigate to your desktop or project folder:

```
cd Desktop
```

or

```
cd Documents/LS100
```

2. Create a project folder (optional):

```
mkdir LS100_YOLO_Project
```

```
cd LS100_YOLO_Project
```

3. Create the virtual environment named `yolo_venv`:

```
python3.12 -m venv yolo_venv
```

Step 5: Activate the environment

You need to "get inside" this environment so your codes use the tools and packages inside it.

- **macOS/Linux:**

```
source yolo_venv/bin/activate
```

- **Windows:**

```
yolo_venv\Scripts\activate
```

Check: Your command prompt should now show (yolo_venv) at the beginning.

Step 6: Install required packages

We will install `ultralytics` (which contains YOLOv8) along with other standard data science tools.

Type the following command and press Enter:

```
pip install ultralytics opencv-python pandas numpy tqdm matplotlib seaborn jupyter ipykernel
```

- **ultralytics:** The library used to run YOLO.
- **opencv-python:** A tool for processing images and video.
- **jupyter & ipykernel:** Tools tailored to run Python code in a "Notebook" format.

Step 7: Register the environment with Jupyter

After installation, register the newly created environment so Jupyter Notebook can see it.

```
python -m ipykernel install --user --name=yolo_venv --display-name="LS100_YOLO"
```

- `--name:` The internal identifier (system name).
- `--display-name:` The name you will see in the Jupyter menu (you can change this name).

Step 8: Launch Jupyter Notebook

Launch Jupyter Notebook from the terminal. It will open in your web browser.

```
jupyter notebook
```

Step 9: Open a notebook

A new page will open in your web browser, showing your file directory.

- Click the **"New"** button (or "New ▼") on the right side.
- Select the kernel you just created: **LS100_YOLO**.

Step 10: Select your virtual environment kernel

If you open an existing notebook, ensure the correct kernel is selected.

- Look at the top right corner of the notebook interface.
- If it says "Python 3 (ipykernel)" or something else, click it and select **LS100_YOLO** from the list.

Step 11: A Quick Introduction to Jupyter Notebook

Before we write our first YOLO code, let's briefly look at how Jupyter Notebook works. A notebook is an interactive document made up of rectangular blocks called **cells**.

There are two main types of cells you will interact with:

- **Code Cells:** This is where you will type your Python commands. You can recognize a code cell because it has `In [ ] :` next to it on the left side.
- **Markdown Cells:** This is where you can write standard text, notes, or explanations for yourself. It does not run as computer code.

How to run a cell:

To execute the Python code inside a cell, or to format the text you typed in a Markdown cell:

1. Click anywhere inside the cell to select it.
2. Press **Shift + Enter** on Windows (or **Shift + Return** on a Mac) on your keyboard. Alternatively, you can click the "Run" button (the play icon ►) in the toolbar.

How to tell if a cell is running:

When you run a code cell, look at the square brackets `[ ]` on the left side:

- `In [*]`: An asterisk means the computer is currently busy processing that code.
- `In [1]`: A number means the code has finished running. The number tells the order in which the cells were executed.

When you run a Code cell, the computer processes your instructions, and any result—like a printed message, a table of data, or a detected image—will appear immediately below that cell!

Getting Started: How to Use YOLO

Now that your environment is ready, copy and run the following code in your Jupyter Notebook to perform your first object detection.

1. Load the Model

First, we import the library and load a pre-trained model. We will use `yolov8n.pt`, which is the "Nano" version; it is the smallest and fastest model, perfect for laptops. In a code cell of your notebook, type the following and then press "shift + return" to run the code in that cell.

```
from ultralytics import YOLO

# Load the model. This step will download the model automatically the first
time you run this.

# Load a standard model for detecting objects (like cars, dogs, etc.)
detection_model = YOLO('yolov26n.pt')

# Load a specialized model for detecting human body keypoints (Pose
Estimation)
pose_model = YOLO('yolov26n-pose.pt')
```

2. Make a Prediction

Example A: Basic Object Detection

This identifies what is in the image and draws a box around it. You can run the model on an image from the internet or one you have uploaded to your folder.

```
# Run inference on an image (using a sample URL here)
results = detection_model('https://ultralytics.com/images/bus.jpg')

# The 'results' variable now contains the detection data (coordinates, labels, etc.)

# Show the results
for result in results:
    result.show() # This will open a pop-up window with the image
```

Example B: Detecting Body Keypoints (Pose Estimation)

Instead of just drawing a box around a person, this version detects specific "keypoints" like eyes, ears, shoulders, elbows, and knees.

```
# Run pose estimation on an image of a person
```

```
# You can replace this URL with a path to a photo on your computer
pose_results = pose_model('https://ultralytics.com/images/human.jpg')

# Show the results (skeleton and keypoints)
for result in pose_results:
    result.show()
```

Understanding the Keypoints:

When you run the Pose model, YOLOv8 identifies 17 standard body points. In your code, you can actually see the coordinates (where they are on the screen) by adding this line:

```
# This prints the X and Y coordinates for every detected body part
print(pose_results[0].keypoints.xy)
```

What just happened?

- **Labels:** The model identified objects (like "bus", "person") and assigned them a specific class.
- **Confidence:** The numbers next to the labels (e.g., 0.85) show how confident the model is (85% sure).
- **Bounding Boxes:** The rectangles show exactly where the objects are located in the image.
- **Skeleton:** For the pose model, the model automatically connected the dots (keypoints) to draw a human skeleton.